CS1200: Intro. to Algorithms and their Limitations	Prof. Anurag Anshu

Lecture 19: NP-completeness

Harvard SEAS - Spring 2026 *2026-13-04*

1 Announcements

- Anurag's OH Wed 12:45-1:45 pm
- Next SRE Mon April 20.

Recommended Reading:

- Hesterberg–Vadhan 18
- MacCormick, 12.0 - 12.3, 13

2 Complexity Classes

Following up on the above ideas, we define the following *complexity classes*.

Definition 2.1 (complexity classes)**.** For a function $T : \mathbb{N} \to \mathbb{R}^+$ with $T(n) \geq n+2$,

- $\mathsf{TIME}_{\mathsf{search}}(T(n))$ is
- $\mathsf{TIME}(T(n))$ is
- (Polynomial time)

$$\mathsf{P}_{\mathsf{search}} = \qquad\qquad \mathsf{P} = \qquad\qquad .$$

- (Exponential time)

$$\mathsf{EXP}_{\mathsf{search}} = \qquad\qquad \mathsf{EXP} = \qquad\qquad .$$

Note that $\mathsf{P}_{\mathsf{search}}$ and $\mathsf{EXP}_{\mathsf{search}}$ would be the same if we replace Word-RAM programs with algorithms described using any strongly Turing-equivalent model, like Turing Machines.

The algorithms that we have covered this semester demonstrate that a number of problems are in $\mathsf{P}_{\mathsf{search}}$. For some other problems, we only know that they are in $\mathsf{EXP}_{\mathsf{search}}$. Can we prove that they are not in $\mathsf{P}_{\mathsf{search}}$?

The following seems to give us some hope:

Theorem 2.2.

Unfortunately, we do not know how to prove that many of the problems we care about (like the ones mentioned above) are in $\mathsf{EXP}_{\mathsf{search}} \setminus \mathsf{P}_{\mathsf{search}}$.

So our state of knowledge is as follows:

To get a handle on why these problems seem hard, we will introduce another complexity class.

3 $\mathsf{NP}_{\mathsf{search}}$

Roughly speaking, $\mathsf{NP}_{\mathsf{search}}$ consists of the computational problems where valid outputs can be *verified* in polynomial time. This is a very natural requirement; what's the point in searching for something if we can't recognize when we've found it? See Figures 1 for an illustrative example; note that verifying a solution appears to be much easier than finding one.

Formally:

Definition 3.1. A computational problem $\Pi = (\mathcal{I}, \mathcal{O}, f)$ is in $\mathsf{NP}_{\mathsf{search}}$ if the following conditions hold:

1. All valid outputs are of polynomial length:

Figure 1: Can you find a cat?

2. All valid outputs are verifiable in polynomial time:

Let's see some examples of problems in $\mathsf{NP}_{\mathsf{search}}$.

Proposition 3.2. Satisfiability, Graph Coloring, *and* Independent Set–Threshold Search *are all in* $\mathsf{NP}_{\mathsf{search}}$.

Proof. For Satisfiability, we have

For Graph Coloring, we have

For Independent Set–Threshold Search, we have

□

As a *non*-example, consider Independent Set–Optimization Search, where

The following proposition shows that every problem in $\mathsf{NP}_{\mathsf{search}}$ can be solved in exponential time.

Proposition 3.3. $\mathsf{NP}_{\mathsf{search}} \subseteq \mathsf{EXP}_{\mathsf{search}}$.

Proof.

□

So now our diagram of complexity classes is the following:

Remark 3.4. On $\mathsf{P}_{\mathsf{search}}$ vs $\mathsf{NP}_{\mathsf{search}}$: Somewhat counterintuitively, $\mathsf{P}_{\mathsf{search}} \not\subseteq \mathsf{NP}_{\mathsf{search}}$. This due to artificial examples, but most of the natural problems in $\mathsf{P}_{\mathsf{search}}$ are also in $\mathsf{NP}_{\mathsf{search}}$.

Remark 3.5. Every problem in $\mathsf{NP}_{\mathsf{search}}$ has a corresponding decision problem (deciding whether or not there are any valid outputs). The class of such decision problems is called NP. We will discuss the class NP more next lecture.

Remark 3.6. The 'N' in $\mathsf{NP}_{\mathsf{search}}$ and NP stands for 'nondeterministic,' because problems in these classes can be solved by "nondeterministically guessing" a solution and efficiently verifying that it is valid. We will not make precise this notion of nondeterministic computation.

4 Polynomial-Time Reductions

Although we do not know how to prove that many problems we care about are not in $\mathsf{P}_{\mathsf{search}}$, it turns out that we can get a much better understanding of their complexity via *reductions*. Since we have adopted "polynomial time" as our coarse notion of "efficiently solvable," it makes sense to do the same for reductions:

Definition 4.1. For computational problems Π and Γ, we write $\Pi \leq_p \Gamma$ if

Using polynomial-time reductions to compare problems fits nicely with the study of the classes $\mathsf{P}_{\mathsf{search}}$ and P, since they are "closed" under such reductions:

Lemma 4.2. *Let Π and Γ be computational problems such that $\Pi \leq_p \Gamma$. Then:*

1.

2.

Proof. 1.

2. Contrapositive of Item 1

□

The vast majority of reductions we have seen in this book so far have been polynomial-time reductions. Typically, we have used these for *positive* results:

However, Item 2 shows that we can also use reductions for *negative* results, to give evidence that problems are not in $\mathsf{P}_{\mathsf{search}}$. As always, *the direction of the reduction is crucial!*

Another very useful feature of polynomial-time reductions is that they compose with each other:

Lemma 4.3. *If* $\Pi \leq_p \Gamma$ *and* $\Gamma \leq_p \Theta$ *then* $\Pi \leq_p \Theta$.

This follows by similar reasoning to Lemma 4.2.

5 $\mathsf{NP}_{\mathsf{search}}$-completeness

Although it is widely conjectured, we unfortunately do not know how to prove that $\mathsf{NP}_{\mathsf{search}} \not\subseteq \mathsf{P}_{\mathsf{search}}$. As we will see in the next lecture, this is an equivalent formulation of the famous P vs. NP problem, considered one of the most important open problems in computer science and mathematics. However, even without resolving the P vs. NP conjecture, we can give strong evidence that problems are not solvable in polynomial time by showing that they are $\mathsf{NP}_{\mathsf{search}}$-*complete*:

Definition 5.1 (NP-completeness, search version)**.** A problem Γ is $\mathsf{NP}_{\mathsf{search}}$-*complete* if:

1.

2.

We can think of the NP-complete problems as the "hardest" problems in NP. Indeed:

Proposition 5.2. *Suppose* Γ *is* $\mathsf{NP}_{\mathsf{search}}$*-complete. Then* $\Gamma \in \mathsf{P}_{\mathsf{search}}$ *iff* $\mathsf{NP}_{\mathsf{search}} \subseteq \mathsf{P}_{\mathsf{search}}$.

Proof.

□

In other words, if any $\mathsf{NP}_{\mathsf{search}}$-complete problem is in $\mathsf{P}_{\mathsf{search}}$, then all problems in $\mathsf{NP}_{\mathsf{search}}$ are in $\mathsf{P}_{\mathsf{search}}$.

6 $\mathsf{NP}_{\mathsf{search}}$-complete problems

Remarkably, there not only exist $\mathsf{NP}_{\mathsf{search}}$-complete problems, but some of them are quite natural. The first one we consider is SAT:

Theorem 6.1 (Cook–Levin Theorem)**.** SAT *is* $\mathsf{NP}_{\mathsf{search}}$*-complete.*

This can be interpreted as strong evidence that SAT is not solvable in polynomial time. If it were, then *every* problem in $\mathsf{NP}_{\mathsf{search}}$ would be solvable in polynomial time. We will take the Cook–Levin Theorem on faith for now, and may sketch a proof later.

Once we have one $\mathsf{NP}_{\mathsf{search}}$-complete problem, we can get others via reductions from it.

Theorem 6.2. 3SAT *is* $\mathsf{NP}_{\mathsf{search}}$*-complete.*

Proof. The proof follows in two steps.

1. 3SAT is in $\mathsf{NP}_{\mathsf{search}}$:

2. 3SAT is $\mathsf{NP}_{\mathsf{search}}$-hard: Since every problem in $\mathsf{NP}_{\mathsf{search}}$ reduces to SAT (by Theorem 6.1), all we need to show is SAT $\leq_p$ 3SAT (since reductions compose, by Lemma 4.3).

The reduction algorithm from SAT to 3SAT has the following components. First, we give an algorithm R which takes a SAT instance φ to a 3SAT instance φ'.

$$\text{SAT instance } \varphi \xrightarrow{\text{polytime R}} \text{3SAT instance } \varphi'$$

Then we feed the instance φ' to our 3SAToracle and obtain a satisfying assignment α' to φ' or $\perp$ if none exists. If we get $\perp$ from the oracle, we return $\perp$, else we transform α' into a satisfying assignment to φ using another algorithm S.

$$\text{SAT assignment } \alpha \xleftarrow{\text{polytime S}} \text{3SAT assignment } \alpha'$$

Pictorially:

Algorithm R: The intuition behind this algorithm is that when we have a clause $(\ell_0 \vee \ell_1 \vee ... \vee \ell_{k-1})$ in the SAT instance φ (with too-large width $k > 3$), we want to break it into multiple clauses of width ≤ 3.

```
R(φ):
Input      : A CNF formula φ
Output     : A CNF formula φ' with each clause of width 3
φ' = φ
i = 0
while φ' has a clause C = (ℓ_0 ∨ ... ∨ ℓ_{k-1}) of width k > 3 do
    Remove C
    Add clauses _____________________________________
    i = i + 1
return φ'
```

Algorithm 6.1: R

Note that φ' is *not* an equivalent formula to φ. While φ is on variables $z_0, \ldots z_{n-1}$, the formula φ' is on variables $z_0, \ldots z_{n-1}, y_0, \ldots y_{t-1}$, where t is the number of iterations of the while loop.

Algorithm S: Given an assignment $\alpha' = (\alpha_0, \ldots, \alpha_{n-1}, \beta_0, \ldots, \beta_{t-1})$ to φ', the algorithm simply takes the first n bits, i.e. $\alpha = (\alpha_0, \ldots, \alpha_{n-1})$.

Next we consider the runtime and correctness of the overall reduction algorithm.

Runtime of the reduction algorithm: We first consider the runtime of the algorithm R:

Then, we consider the runtime of the algorithm S, which is simply $O(n)$. Overall, the runtime of the reduction algorithm is $O(nm)$.

Proof of correctness: We need to show that if φ is satisfiable, then the reduction algorithm produces a satisfying assignment, and that if φ is unsatisfiable, the reduction algorithm outputs $\perp$. This proof relies on the following two claims.

Claim 6.3. *If φ is satisfiable then $\varphi' = R(\varphi)$ is satisfiable.*

Proof sketch.

$\square$

Claim 6.4. *If α' satisfies $\varphi' = R(\varphi)$, then $\alpha = S(\alpha')$ also satisfies φ.*

Proof sketch.

□

To finish the correctness proof, suppose φ is satisfiable. Then from Claim 6.3, φ' is also satisfiable. The 3SAT oracle returns a satisfying assignment α', which is turned into a satisfying assignment for φ via the algorithm S (Claim 6.4). If φ is unsatisfiable, then by Claim 6.4, φ' is also unsatisfiable. In this case, the 3SAT oracle returns $\perp$, so the reduction algorithm also returns $\perp$.

This completes the proof that 3SAT is $\mathsf{NP}_{\mathsf{search}}$-complete.

7 Mapping Reductions

The usual strategy for proving that a problem Γ in $\mathsf{NP}_{\mathsf{search}}$ is also $\mathsf{NP}_{\mathsf{search}}$-hard (and hence $\mathsf{NP}_{\mathsf{search}}$-complete) follows a structure similar to the proof of Theorem 6.2.

1. Pick a known $\mathsf{NP}_{\mathsf{search}}$-complete problem Π to try to reduce to Γ.
2. Come up with an algorithm R mapping instances x of Π to instances $R(x)$ of Γ.
3. Show that R runs in polynomial time.
4. Show that if x has a valid output, then so does $R(x)$.
5. Conversely, show that if $R(x)$ has an answer, then so does x. Moreover, we can transform valid answers to $R(x)$ into valid answers to x through a *polynomial time* algorithm S.

Reductions with the structure outlined above are called *polynomial-time mapping reductions*, and they are what are typically used throughout the theory of NP-completeness. A formal definition is given in the textbook.